The SAT problem

Define

$$\mathbb{B} = \{0, 1\}$$

We want to answer the question, for a boolean function $f : \mathbb{B}^n \to \mathbb{B}$ which consists purely of the conjunction $\wedge$, disjunction $\vee$, and negation operator $\neg$; we want to determine:

$$\exists \mathbf{x} \in \mathbb{B}^n \text{ such that } f(\mathbf{x}) = 1$$

Motivating the SAT problem through reductions to SAT

Searching for tautologies

Suppose we want to determine whether the boolean function $g : \mathbb{B}^n \to \mathbb{B}$ is a tautology, that is, we want to determine whether:

$$\forall \mathbf{x} \in \mathbb{B}^n, g(\mathbf{x}) = 1$$

We want to make this in the form of the SAT problem.

To do this:

$$\neg \exists \mathbf{x} \in \mathbb{B}^n, g(\mathbf{x}) = 0$$

We define:

$$f(\mathbf{x}) = \neg g(\mathbf{x})$$

Thus, our problem becomes:

$$\neg \exists \mathbf{x} \in \mathbb{B}^n, f(\mathbf{x}) = 1$$

So we just solve the SAT problem and then negate the answer.

Determining equivalences

Suppose that we have two boolean functions $g : \mathbb{B}^n \to \mathbb{B}$ and $h : \mathbb{B}^n \to \mathbb{B}$.

The question is:

$$\forall \mathbf{x} \in \mathbb{B}^n, g(\mathbf{x}) == h(\mathbf{x})$$

This means that we can define:

$$f(\mathbf{x}) = \neg\left(g(\mathbf{x}) == h(\mathbf{x})\right)$$

And then ask the question:

$$\forall \mathbf{x} \in \mathbb{B}^n, f(\mathbf{x}) = 0$$

And so:

$$\neg\exists \mathbf{x} \in \mathbb{B}^n, f(\mathbf{x}) = 1$$

Note, we see that for 2 boolean variables $p, q \in \mathbb{B}$:

$$p = q \iff p \oplus q = 0$$

Therefore, we can even define:

$$f(\mathbf{x}) = g(\mathbf{x}) \oplus h(\mathbf{x})$$

Note that XOR operator $\oplus$ can be formed from just the $(\wedge, \vee, \neg)$ symbols, that is:

$$p \oplus q = (p \wedge \neg q) \vee (\neg p \wedge q)$$

We can place it in CNF form:

$$p \oplus q = (\neg(\neg p \vee q)) \vee (\neg(p \vee \neg q)) = (\neg p \vee q) \wedge (p \vee \neg q)$$

K-Coloring problem to SAT

Let's first define the K-coloring problem for graphs.

A graph can be defined by the set of edges and verticies.

That is, first, we have a set of vertices V, and then we have a set of edges E such that all elements in the set composes of another sets going from one vertex to another vertex in V.

The K-coloring first assigns k colors, say in the set C such that $|C| = k$.

Essentially, we want to determine a coloring function $c : V \to C$, such that:

$$\forall (i, j) \in V; c(i) \neq c(j)$$

To turn this into SAT, we essentially want to create a boolean function f which will be satisfiable only if there's a valid color assignment to each vertices from the k colors:

Create boolean variables:

$$C_{c,v}$$

Where we override the definition of c to be simply be an element:

$$c \in C$$

And:

$$v \in V$$

First of all, we encode the fact that for each $v \in V$, we only assign one color.

To do this, we state that to assign exactly one color to a vertex:

- every vertex must have at least one color assignment
- every vertex must have at most one color assignment

To say that every vertex must have at least one color, we essentially want:

$$\forall v \in V, \bigvee_{c \in C} C_{c,v}$$

To say that every vertex must have at most one color, we want something like:

- for every vertex, for every pair of boolean variables representing that vertex, it can never be that both of them are true.

That is, we get the truth table:

p	q	desired
0	0	1
0	1	1
1	0	1
1	1	0

So the desired boolean function is $\neg(p \wedge q)$

So essentially:

$$\forall v \in V, \bigwedge_{i,j \in C \wedge i<j} (\neg C_{i,v} \vee \neg C_{j,v})$$

We must also constraint that no neighboring vertices should have the same color. That is to say, if we one of the boolean variable with the same c is true (we assigned some color to a one of the vertex), it we want the whole expression to be false when the other guy is also true.

p	q	desired
0	0	1
0	1	1
1	0	1
1	1	0

So it's going to be $\neg(p \wedge q)$ again.

$$\forall (u, v) \in E, \forall c \in C, \neg C_{c,u} \vee \neg C_{c,v}$$

Sudoku to SAT

It is possible to encode sudoku into k-coloring (standard sudoku would be 9-coloring) but we can also do it directly.

We want to define a boolean function f such that it will be satisfiable only if the sudoku puzzle have a solution.

We have $r \in R$ columns, $c \in C$ rows and $d \in D$ digits.

Essentially, we probably want the digit set $|D|$ to be of a square number, and then we want $|R| = |C| = |D|$.

But in either case, suppose we already have our parameters for the sudoku board.

So to do this, we define boolean variables as such:

$$A_{r,c,d}$$

We now impose the following constraints:

- for every $(r, c) \in R \times C$, we want only one digit in each (r, c), this can be broken down into 2 constraints:
  - for every $(r, c) \in R \times C$, we want at least one of the $A_{r,c,d}$ to be true, that is

$$\forall (r, c) \in R \times C, \exists d \in D, A_{r,c,d}$$

  - for every $(r, c) \in R \times C$, we want at most one of the $A_{r,c,d}$ to be true, that is, for every unordered pair $(i, j) \in D^2$ (need to constraint that they're not identical, and doesn't care about order), we never want both of them to be true, that is:

$$\forall (r, c) \in R \times C, \forall (i, j) \in D^2, (i < j) \land (\neg A_{r,c,i} \lor \neg A_{r,c,j})$$

- now that we assign uniquely only one digit per (r, c), we now need to constraint that fact that, for every row, they need to be unique, for every column, they need to be unique, and also unique for the sub-square.

I'll just define a set constructor:

$$S(r, c)$$

Which gives you the set of coordinates of a sub-square that the coordinate (r, c) belongs in.

So to encode that each digits in every row and columns should be different:

$$\forall r \in R, \forall (i, j) \in C^2, \forall d \in D, (i < j) \land (\neg A_{r,i,d} \lor \neg A_{r,j,d})$$

We do the same for all rows, and then we do the same for all sub-squares using the sub-square constructor $S(u, v)$.

We unravel the existential and universal quantifier into the disjunction and conjunction operators.

Also, as long as you have universal quantifiers in the outer loop, existential quantifiers in the inner loop, and inner expression defined without the conjunction $\land$ operator, it should be in CNF form.

Plan to implement a SAT solver

The plan is we first build the generic data structure and utilities that any SAT solver should have and share, and then modify the core of the algorithm to optimize it.

If our input is already in CNF, we don't need to construct any tree structure, because we know structure of the operators already.

data structure

So first, our boolean variables will be uniquely identified by integers, with a negative sign to indicate the negation sign.

We will essentially have a 2d array, for example:

```
expression = [
  [-1, 2, 3],
  [2, -3, 4, 5],
]
```

In the inner array, it expresses variables to OR into one another, and then the outer loop expresses variables to AND into one another.

Now for assignment, we use `-1` for `false` assignment `0` for unassigned, and `1` for `true` assignment.

For example:

```
assignment = [-1, 1, 0, 1, -1]
```

Where each index corresponds to one of the variables in our expression. Also, note that if we to evaluate, it's going to be $O(m)$ where m is total size of the expression, because we can access the assignment in O(1) time.

We can define an `evaluate` function:

```
evaluate(expression, assignment)
```

Bruteforce

Essentially, we can just do bruteforce by setting the assignment all to false, incrementing it one by one and then evaluate it.

We can also construct some sort of uniform shuffle function such that we select different assignments randomly, so that the average running time would be:

$$O(\rho 2^n)$$

Where ρ is going to be the proportion of $\frac{N(\text{false})}{2^n}$, where $N(\text{false})$ is essentially the total amount of `false` results we get from the boolean expression.

Let's see how we can get this uniform shuffle function, without having to do `random.shuffle(list(range(2**n)))`

Essentially, we want a function f that has a domain of $D = \{0, 1, ..., n\}$ and a range of D also, such that it's bijective, and such that it shuffles it in a sort of uniform pseudo random manner. I suppose if we want this function to be deterministic, you should also input a seed.

It means that:

$$f : D \times \mathbb{R} \to D$$

And it's bijective when we fix $r \in \mathbb{R}$.

The reason why we want to construct such a function is because, we don't want to store $O(2^n)$ assignments

One of the ways to construct such a function that's bijective is to use modulo arithmetic, however, it's not going to be random because it's just going to translate.

I think we can modify some well-known hash function to do this.

Also, this problem is called pseudo random permutation.

I suppose I can look into Feistel networks

DPLL